Data Science Assessment – Task 2 Report

Overview
This document addresses **Task 2** of the Data Science assessment

Task overview:
Computer Vision classification challenge: Use a sample of images from the swimming-pool dataset to develop a model that classifies whether an image contains a swimming pool or not. Use the provided labels to validate your model.

Initial thoughts.
Training effective image recognition models requires substantial computational resources. Combined with time constraints, this means we will initially adopt a small-scale approach to better understand how to develop a robust model.

Data Overview
- **Found** 4,643 *"yes"* images (swimming pools)
- **Found** 2,400 *"no"* images (no swimming pools)

Models overview
- **Initial model** – basic model to check model training and provisional results
- **Advanced model** – integrate learnings from initial model and advancements into a more advanced model
- **Final model** – large scale development of the best model.

Results metrics:
- **Precision:** The proportion of predicted positives that are actually correct.
- **Recall:** The proportion of actual positives that the model correctly identifies.
- **F1-score:** The harmonic mean of precision and recall, balancing both metrics into a single measure.

Output:
An image recognition model that recognises aerial images with / without pools.

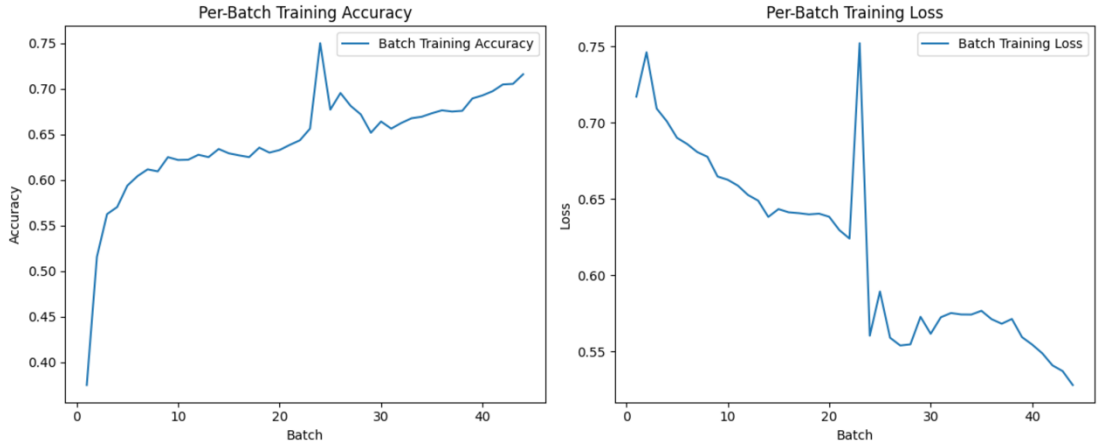
Initial Model

Python code: *Task2_Intial model.py*
Model Description
An initial convolutional neural network (CNN) image recognition model was developed from scratch using 10% of the available data to evaluate training times, convergence behaviour, and provisional classification performance.

Experiment 1 – Initial Model
Training:
- Model: CNN
- Epochs: 2
- TRAIN_PERCENT: 0.10 (10%)
- VAL_PERCENT: 0.05 (5%)
    - Total training samples: 704

- Total validation samples: 352

Training Loss:



Results:

Class	Precision	Recall	F1-score
0	0.91	0.35	0.51
1	0.75	0.98	0.85

- **Accuracy:** 0.77 (352 samples)
- **Macro Avg:** Precision 0.83, Recall 0.67, F1-score 0.68
- **Weighted Avg:** Precision 0.80, Recall 0.77, F1-score 0.73

Data Loading Times:

Phase	Duration (seconds)
Training	1,401 ~ 23 mins
Validation	531

Discussion:

From the convergence plots, it is evident that the model was learning: training loss decreased, and accuracy increased over batches. However, the model showed strong bias toward the majority class.

Training a CNN on image data is computationally expensive: using only 10% of the data, a simple architecture, and 2 epochs, training and validation took approximately 32 minutes.

Results Interpretation:

- **Class imbalance** was a clear factor:
  - ~66% of validation data was class 1 ("yes")
  - The model predicted class 1 most of the time, leading to:
    - Very high recall (0.98) for class 1 (almost all "yes" images detected)
    - Low recall (0.35) for class 0 (many "no" images missed)
  - Precision for class 0 was high (0.91) because the few predictions it made for "no" were often correct.

This is a typical symptom of class imbalance, where the model overfits to the majority class.

Conclusion:

This was a promising first attempt demonstrating:

- The model can learn meaningful patterns
- Training infrastructure is operational

Next steps:

- Balance the dataset
- Experiment with more advanced model architectures

Advanced Model

Python Code: *Task2_Advanced model.py*

Description

A more advanced model was developed to address the limitations identified in the initial experiment. Improvements included:

- Balancing class counts
- Integrating MobileNetV2, a pre-trained convolutional network, via Keras

---

Experiment 1 – Balanced Data and MobileNetV2

Training:

- Epochs: 2
- Training samples: 400 (200 "yes", 200 "no")
- Validation samples: 200 (100 "yes", 100 "no")

Results:

Class	Precision	Recall	F1-score
No Pool	0.81	0.83	0.82
Pool	0.82	0.80	0.81

- **Accuracy:** 0.81 (200 samples)
- **Macro Avg:** Precision 0.82, Recall 0.81, F1-score 0.81
- **Weighted Avg:** Precision 0.82, Recall 0.81, F1-score 0.81

Summary of Durations:

Phase	Duration (seconds)
Total Training Time	1,696.47 ~ 28 mins
Validation Download Time	296.09

Conclusion:

Despite using fewer samples (400 vs. 704 in the initial model), this model achieved higher validation accuracy. Training time was slightly longer due to the complexity of MobileNetV2 (~28 minutes). This demonstrates the benefits of using pre-trained architectures even with modest datasets, however, also demonstrates the added computational expense of this mode advanced model.

Next steps:

- Increase the volume of training data
- Extend training epochs

Final Model

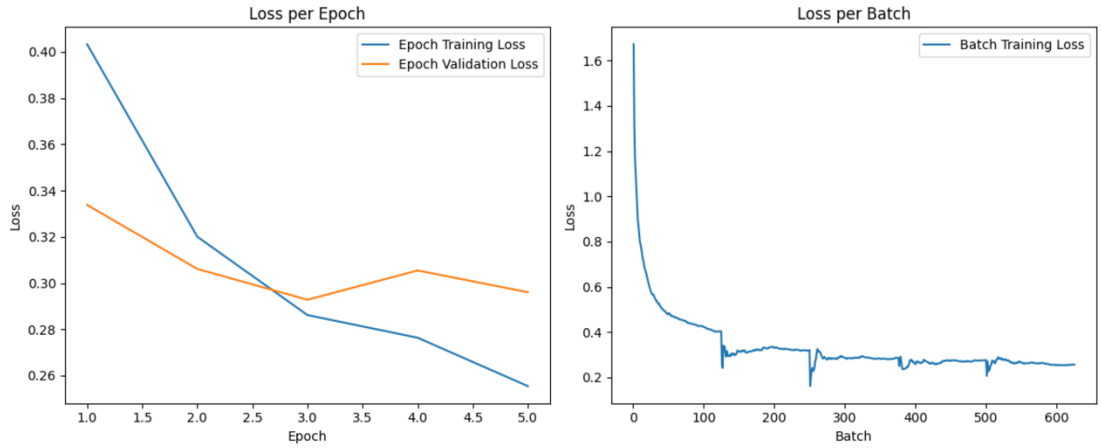
Python Code: *Task2_Advanced model.py*

Experiment 2 – Larger Dataset and Extended Training

Training:

- Epochs: 5
- Training samples: 4,000 (2,000 "yes", 2,000 "no")
- Validation samples: 800 (400 "yes", 400 "no")

Training Loss Plot:



Results:

Class	Precision	Recall	F1-score
No Pool	0.92	0.85	0.88
Pool	0.86	0.92	0.89

- **Accuracy:** 0.89 (800 samples)
- **Macro Avg:** Precision 0.89, Recall 0.89, F1-score 0.88
- **Weighted Avg:** Precision 0.89, Recall 0.89, F1-score 0.88

Summary of Durations:

Phase	Duration
Total Training Time	35,747.77 ~ 10 hours
Validation Download Time	645.79

Detailed Metrics Interpretation:

- **Precision**
  - No Pool: 92%
  - Pool: 86%
- **Recall**
  - No Pool: 85%
  - Pool: 92%
- **F1-Score**
  - Balanced across classes (0.88–0.89)
- **Accuracy**
  - 89% overall

This indicates a strong, well-balanced classifier.

Conclusion:

This final model demonstrates significantly improved accuracy and balanced performance across classes. While training time increased substantially (~9.9 hours), the results show readiness to scale further and justify investing in full dataset training.

Next Steps:

- Continue expanding data coverage
- Increase training epochs
- Explore hyperparameter tuning
- Consider ensembling or fine-tuning further advanced architectures